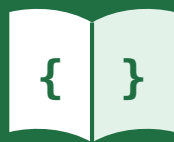

Module 24 — Spring AI

Exercises

Spring Boot Edu



2026-09-25

Contents

How to Work	2
Exercise 1 — A Book Suggestion Assistant (Easy)	2
Exercise 2 — Customer FAQ with RAG (Medium)	2
Exercise 3 — Where Is My Order? (Medium)	3
Extra Challenge (Optional)	3

How to Work

1. The starter code is in `modules/24-spring-ai/exercise/`. Find the TODO comments.
2. The tests use a fake `ChatModel` and a keyword `EmbeddingModel` (lesson section 3.8), and PostgreSQL with `pgvector` from `Testcontainers`: Docker must run, but no language model is needed.

```
./mvnw -Pexercises -pl modules/24-spring-ai/exercise -am test
```

3. The exercise is done when all tests are green. Then try your code with the real model if you like (the lesson's `spring-boot:run` shows how Ollama is started).
4. If you get stuck, read the hints first, then the solution in `modules/24-spring-ai/solution/`.

Exercise 1 — A Book Suggestion Assistant (Easy)

Goal: a customer names their interests, and the assistant suggests a number of books, as a list of records.

Tasks (`exercisel.SuggestionAssistant.suggest`):

- TODO 1a — the user message comes from the template `prompts/suggest.st`, with the parameters `interests` and `count`.
- TODO 1b — the answer becomes a `List<BookSuggestion>`.

Hints:

- Lesson section 3.3: `.user(user -> user.text(template).param(...))`.
- A list needs the generic type: `.entity(new ParameterizedTypeReference<List<BookSuggestion>>() { })`.

Acceptance criteria: both tests in `ExerciselTest` pass.

Estimated time: 20 minutes

Exercise 2 — Customer FAQ with RAG (Medium)

Goal: questions about shipping, returns and payment are answered from the shop's FAQ (`src/main/resources/faq/*.md`), not from the model's imagination.

Tasks (`package exercise2`):

- TODO 2a (`FaqIngestion.ingest`) — read every FAQ file as a document with the meta-data `collection = faq`, add them to the vector store and return how many were added. Ingesting twice must not create duplicates.

- TODO 2b (FaqAssistant) — the best-matching FAQ entry (only one) must be part of every prompt.

Hints:

- Lesson section 3.6: `TextReader`, `getCustomMetadata()`, `vectorStore.delete("collection == 'faq'")`.
- The texts are short: no splitter needed.
- `QuestionAnswerAdvisor.builder(vectorStore).searchRequest(SearchRequest.builder().topK(1))`

Acceptance criteria: all 3 tests in `Exercise2Test` pass.

Estimated time: 30 minutes

Exercise 3 — Where Is My Order? (Medium)

Goal: the support assistant can look up the status of an order with a tool.

Tasks (package `exercise3`):

- TODO 3a (`OrderTools.orderStatus`) — make the method a tool: a description for the model (what it returns, when to use it) and a description of the parameter (the order ID, for example A-1001).
- TODO 3b (`SupportAssistant`) — the model may call the tools of `OrderTools`.

Hints:

- Lesson section 3.5: `@Tool(description = ...)`, `@ToolParam(description = ...)`, `.defaultTools(...)`.
- `Exercise3Test` checks the tool definition with `ToolCallbacks.from(tools)`: the name, the description and the example in the parameter schema.

Acceptance criteria: all 3 tests in `Exercise3Test` pass.

Estimated time: 20 minutes

Extra Challenge (Optional)

Add a tool that cancels an order. It must not cancel directly: it returns a confirmation code, and only a second call with this code cancels. Write down which prompt injection this design protects against.